Contrôle d'admission stigmergique et de quorum : guide du praticien

`moderation-pipeline-prototype`*

Août 2026

1 Quand ce guide s'applique

Les trois conditions suivantes, réunies :

1. Votre système possède une voie rapide et peu coûteuse, ainsi qu'un recours coûteux, dans le pire cas exponentiel (ou autrement onéreux), vers lequel basculent les dossiers dès que la voie rapide ne tranche pas.
2. L'admission au recours coûteux est aujourd'hui subordonnée à peu de chose : dès qu'un dossier reçoit un premier signal de confirmation plausible, il passe.
3. Un adversaire peut fabriquer ce premier signal de confirmation à bas coût, en volume, en une seule salve, sans avoir à soutenir quoi que ce soit ni à coordonner des identités dans la durée.

Le déploiement de référence est un pipeline de modération de contenu à quatre étages, dont l'étage 2b bascule vers l'énumération, NP-complète, des extensions préférées d'un cadre d'argumentation abstraite (Dung, 1995) lorsque l'extension fondée est vide et que le délai d'escalade d'un dossier a expiré. Rien, ci-dessous, n'est propre aux cadres d'argumentation : la porte traite l'étage coûteux comme un réservoir de ressources opaque, muni d'un signal de libération, et s'insère à toute frontière d'admission correspondant aux trois conditions ci-dessus (files d'attente qui se rabattent sur un solveur exhaustif, escalades de modération vers une révision humaine, accès limité en débit à un appel de modèle coûteux).

Ce que l'on obtient. Une salve de taille quelconque, en un seul pas de temps, issue d'un nombre quelconque d'identités distinctes ou partagées, est bloquée sans condition. Un attaquant patient, soutenu et diversifié en identité n'est pas bloqué, mais son admission est retardée d'un facteur d'environ cinq sous le réglage de référence — la section 4 montre que ce facteur est un cadran, non une constante, et ce qu'il en coûte de le tourner.

2 La porte en trois règles

1. **Décroître à chaque pas, avant toute autre chose.** Chaque dossier porte un score qui diminue d'une fraction fixe ρ à chaque pas de temps discret, qu'il ait reçu ou non un renforcement à ce pas. C'est la règle ordinaire d'évaporation des phéromones ($\tau \leftarrow (1-\rho)\tau$) tirée de l'optimisation par colonies de fourmis, appliquée ici à un score

*Complément à l'article *Une porte d'admission à décroissance pondérée, à escompte d'indépendance des sources et à seuil adaptatif de charge, pour un étage d'argumentation NP-complet* (même dossier), qui porte le modèle formel, les deux propositions et leurs preuves, le corollaire sur la tension d'étalonnage, ainsi que la recherche d'antériorités. Ce guide porte la matière appliquée : parcours commenté, procédure de réglage, dépannage, adaptation.

de pression d'admission plutôt qu'à la force d'une piste. La décroissance s'exécute *en premier*, de sorte qu'un dossier qui devient silencieux commence à perdre du terrain immédiatement, sans période de grâce.

2. **Plafonner la contribution du pas, et escompter les répétitions d'une même source.** À l'intérieur d'un pas, dédupliquer d'abord les sources qui renforcent (une source qui soumet cinq éléments en un pas constitue un seul événement de renforcement, non cinq), puis sommer : 1,0 par source vue pour la première fois au cours des w derniers pas, une petite fraction δ par source revue à l'intérieur de cette fenêtre. Plafonner la somme à un montant fixe γ *après* l'étape de déduplication et d'escompte, jamais avant. Inverser cet ordre est l'erreur d'implémentation la plus lourde de conséquences de toute cette porte — la section 6 la traite en premier, précisément parce qu'elle s'est produite ici.
3. **Relever la barre quand l'étage coûteux est occupé.** Le seuil d'admission n'est pas une constante ; c'est $\theta_0(1 + \kappa L/C)$, où L est le nombre de dossiers occupant l'étage coûteux, à l'échelle du système, à l'instant présent — non un compte cumulatif de tout ce qui a jamais été admis. Se tromper sur cette distinction est le second bogue de la section 6.

3 Parcours commenté

L'implémentation de référence est `src/admission_control.py`, classe `StigmergicQuorumAdmissionControl`. Ce qui suit reprend ce fichier dans l'ordre de lecture, en déplaçant le raisonnement des commentaires de code vers la prose.

3.1 Les paramètres réglables et leurs valeurs par défaut

```
DEFAULT_DECAY_RATE = 0.10          # rho
DEFAULT_TICK_CAP = 1.0             # gamma
DEFAULT_INDEPENDENCE_WINDOW = 3    # w, en pas
DEFAULT_REPEAT_DISCOUNT = 0.05     # delta
DEFAULT_BASE_THRESHOLD = 4.0       # theta_0
DEFAULT_LOAD_SENSITIVITY = 0.5     # kappa
DEFAULT_CAPACITY = 5               # C
```

Ces sept nombres ne sont pas des cadrans indépendants à régler un par un au jugé. Le corollaire 1 de l'article prouve que le rapport $x = \theta_0\rho/\gamma$, évalué à charge nulle, gouverne *à la fois* la facilité d'admission d'un litige authentique et modérément corroboré *et* le nombre de pas soutenus dont un attaquant pleinement saturé a besoin. Il n'existe aucune façon de déplacer x qui avantage un côté sans désavantager l'autre. Les valeurs de référence ci-dessus ont été choisies en mesurant directement ce compromis (non en les réglant à rebours d'un résultat de test souhaité) : elles exigent d'un attaquant environ cinq pas d'effort à saturation complète, tout en gardant la barre franchissable pour un litige recevant un renforcement indépendant à un taux de l'ordre de 40 à 50 % des pas en moyenne. Si le taux de renforcement des litiges authentiques diffère dans votre déploiement, redérivez x pour votre propre population avant de toucher à une seule constante (section 4).

3.2 L'état par dossier et la boucle de pas

```
@dataclass
class CaseAdmissionState:
    score: float = 0.0
    last_reinforced_tick: dict[str, int] = field(default_factory=dict)
    admitted: bool = False
```

```
    admitted_tick: int | None = None
```

`last_reinforced_tick` est indexé par identifiant de cohorte, non par événement : c'est toute la mémoire que le mécanisme garde de qui a récemment renforcé ce dossier, et c'est ce qui rend possible l'escompte d'indépendance sans conserver l'historique complet des événements.

```
def tick(self, t: int, reinforcements: dict[str, list[str]]) -> set[str]:
    newly_admitted: set[str] = set()
    touched = set(self._cases) | set(reinforcements)
    for case_id in touched:
        state = self._cases.setdefault(case_id, CaseAdmissionState())
        if state.admitted:
            continue
        # (1) decroissance d'abord
        state.score *= (1 - self.decay_rate)
        # (2) accumuler la contribution du pas, dedupliquee,
        #     ponderee, plafonnee
        contrib = 0.0
        for cohort in set(reinforcements.get(case_id, [])):
            last = state.last_reinforced_tick.get(cohort)
            fresh = last is None or (t - last) >= self.independence_window
            contrib += 1.0 if fresh else self.repeat_discount
            state.last_reinforced_tick[cohort] = t
        contrib = min(contrib, self.tick_cap)
        state.score += contrib
        # (3) admettre seulement si le seuil adaptatif est franchi
        if state.score >= self.current_threshold():
            state.admitted = True
            state.admitted_tick = t
            self._pending_admitted.add(case_id)
            newly_admitted.add(case_id)
    return newly_admitted
```

Notez `set(reinforcements.get(case_id, []))` dans la boucle sur les cohortes — cet appel à `set()` est l'étape de déduplication, et il s'exécute avant la logique d'escompte, non après. C'est le correctif du premier bogue de la section 6 ; si vous adaptez cette porte à un autre langage ou réécrivez la boucle, placez la déduplication là, et nulle part plus loin.

```
def current_threshold(self) -> float:
    load = len(self._pending_admitted)
    return self.base_threshold * (1 + self.load_sensitivity * load / self.
    capacity)

def release(self, case_id: str) -> None:
    self._pending_admitted.discard(case_id)
```

`release()` n'est pas une plomberie facultative que l'on peut ignorer lors d'une première intégration. Le second bogue de la section 6 montre ce qui arrive si on l'omet.

3.3 L'auxiliaire en forme close

```
def min_ticks_to_admission(threshold, tick_cap, decay_rate) -> float:
    ceiling_fraction = threshold * decay_rate / tick_cap
    if ceiling_fraction >= 1.0:
        return math.inf
    return math.log(1 - ceiling_fraction) / math.log(1 - decay_rate)
```

Cette fonction n'est pas un utilitaire annexe ; c'est la proposition 2 de l'article compagnon rendue exécutable, et elle sert deux fois : une fois pour *concevoir* `base_threshold` en fonction d'un coût attaquant visé, et une fois à l'intérieur de la suite de tests adversariaux, pour vérifier que la porte simulée admet effectivement un attaquant saturé au pas prédit par la forme close. Si vous modifiez `decay_rate`, `tick_cap` ou `base_threshold`, relancez cette fonction avant de relancer la simulation — si les deux divergent de plus d'un pas, quelque chose dans l'implémentation s'est éloigné du modèle qu'elle est censée réaliser.

4 Procédure de réglage

Ne réglez pas les sept constantes indépendamment. Suivez cet ordre :

1. **Fixez ρ à partir de la granularité de vos pas.** ρ fixe la demi-vie de la pression non renforcée, $\ln 2/\rho$ pas. Choisissez d'abord une durée de pas correspondant à la fréquence à laquelle votre système peut plausiblement observer des événements de renforcement (des minutes, non des millisecondes, pour un pipeline de modération humaine), puis choisissez ρ pour que la demi-vie corresponde à la durée pendant laquelle un litige réellement urgent devrait rester « chaud » entre deux renforcements indépendants.
2. **Fixez γ et δ à partir de votre plafond de diversité d'identité.** $\gamma = 1$ (une seule source fraîche par pas sature entièrement le plafond) est la valeur par défaut prudente. δ doit être assez petit pour que le taux de répétition attendu d'une même cohorte ne puisse atteindre le seuil même sur un temps illimité — vérifiez-le avec la proposition 1 : le plafond vaut $\delta\gamma/\rho$, et il doit rester sous θ_0 avec une marge qui vous convient, non tout juste en dessous.
3. **Fixez θ_0 à partir de la tension d'étalonnage, explicitement.** Calculez $x = \theta_0\rho/\gamma$ pour plusieurs valeurs candidates de θ_0 et, pour chacune, calculez à la fois (a) la probabilité qu'un litige authentique et modérément corroboré de votre population franchisse la barre, et (b) $n_{\min}(x)$ obtenu par `min_ticks_to_admission`. Choisissez la valeur pour laquelle (b) est assez grand pour compter opérationnellement (plusieurs tours soutenus d'effort attaquant, non un seul), tout en gardant (a) confortablement franchissable par votre trafic réel. Il n'existe pas de valeur qui maximise les deux à la fois : le corollaire 1 le prouve formellement, inutile d'en chercher une.
4. **Fixez κ et C en dernier, à partir de la capacité réelle de votre étage coûteux.** C doit être votre capacité concurrente réelle pour l'étage coûteux (combien de dossiers il peut traiter à la fois sans dégradation), et κ contrôle avec quelle vigueur la barre monte à mesure que cette capacité se remplit. Commencez à $\kappa = 0{,}5$ et augmentez-le si le test d'ablation (section 4 et l'attaque 5 de l'article) n'étale pas suffisamment les admissions concurrentes pour votre tolérance opérationnelle.
5. **Relancez la suite de tests adversariaux avec vos nouvelles constantes, non seulement le contrôle de cohérence.** Un réglage qui ne vérifie que l'admission des dossiers authentiques, sans relancer les attaques par salve et par inondation soutenue, n'a pas réellement été validé.

5 Liste de contrôle d'intégration

- ☐ Vérifié que la frontière d'admission à protéger correspond bien aux trois conditions de la section 1 — en particulier, que l'« admission » y est aujourd'hui une décision à signal unique, en un seul pas, non déjà limitée en débit par un autre moyen.
- ☐ Identifié votre unité de pas réelle et votre source réelle d'événements de renforcement (ce qui compte concrètement, dans votre système, comme « une cohorte renforçant un dossier » — un signalement, une réfutation, une resoumission).

- ☐ Choisi un proxy d'identité de cohorte et consigné honnêtement sa limite. Ce prototype utilise un identifiant structurel, `rater_creation_cohort` ; quel que soit le proxy retenu, l'escompte d'indépendance de la porte vaut ce que vaut la résistance de ce proxy à l'usurpation (section 8).
- ☐ Câblé `release()` sur l'événement qui marque effectivement la fin du traitement d'un dossier à l'étage coûteux — non sur l'événement d'admission, et non laissé sans appel (section 6, second cas).
- ☐ Redérivé $x = \theta_0 \rho / \gamma$ pour votre propre taux de renforcement des litiges authentiques, plutôt que de recopier telles quelles les constantes de référence (section 4).
- ☐ Relancé les six tests de `tests/test_admission_control.py` avec vos propres constantes, et lu les chiffres imprimés, non seulement le statut de réussite — le taux d'admission du contrôle de cohérence et le retard mesuré imposé à l'attaquant sont les chiffres qui indiquent si le réglage est réellement viable pour votre trafic.
- ☐ Décidé explicitement du sort d'un dossier qui ne franchit jamais le seuil. Cette porte ne possède pas sa propre échappatoire par délai forcé ; si votre système en a besoin (une révision par un médiateur humain après N pas de pression bloquée, par exemple), c'est un mécanisme distinct, superposé, non un défaut caché ici.

6 Dépannage

Les deux lignes ci-dessous sont des bogues réels, découverts en construisant et en testant adversarialement cette porte, non des modes de défaillance hypothétiques.

Symptôme	Cause racine	Correctif
Une inondation d'une même cohorte atteint tout de même le plafond du pas et se fait admettre, alors même que l'escompte de répétition est correctement configuré	Les contributions d'une même cohorte étaient sommées *avant* déduplication à l'intérieur d'un pas, si bien qu'assez d'entrées escomptées issues d'une seule source pouvaient tout de même atteindre le plafond par le seul volume intra-pas	Dédupliquer les cohortes par pas avec un simple appel à `set()` *avant* d'appliquer l'escompte et le plafond, non après
Sous une simulation d'attaque à campagnes multiples et soutenues, le seuil d'admission ne cesse de monter et les dossiers authentiques ultérieurs, pourtant bien corroborés, cessent d'être admis	`release()` n'était jamais appelé, si bien que `_pending_admitted` ne fait que croître, et `current_threshold()` monte de façon monotone, pur artefact de la simulation et non propriété réelle du mécanisme	Modéliser un temps de séjour borné à l'étage coûteux et appeler `release()` à son échéance ; ne pas assouplir l'assertion du test pour masquer un seuil qui aurait dû redescendre

La leçon générale derrière ces deux lignes : tout ce que fait cette porte est sensible au moment *exact* où une étape de mutation s'exécute par rapport au plafond ou au signal de charge qu'elle alimente. Chaque fois que vous touchez à la boucle de pas, revérifiez l'ordre des opérations, pas seulement l'arithmétique.

7 Adapter la porte à une autre frontière d'admission

1. **Identifiez votre proxy de cohorte.** Tout ce qui distingue plausiblement des sources indépendantes convient : âge du compte combiné à un identifiant de grappe de création,

empreinte d'origine réseau, ou, plus faible mais parfois tout ce dont on dispose, simple identifiant utilisateur. L'escompte d'indépendance de la porte vaut ce que vaut la résistance de ce proxy à une usurpation renouvelée à chaque pas — énoncez cette limite explicitement partout où vous déployez cette porte, comme le fait la section 8 ici.

2. **Identifiez votre signal de charge.** Il doit s'agir d'un compte réel, présentement occupé, qui se vide à mesure que des dossiers quittent l'étage coûteux, non d'un compteur cumulatif. Si votre étage coûteux n'a pas de notion naturelle d'« occupation actuelle » (par exemple un calcul coûteux à usage unique plutôt qu'un étage à temps de séjour), la couche de seuil adaptatif à la charge (règle 3 de la section 2) peut ne pas se transposer ; la couche décroissance-plus-plafond (règles 1 et 2) se transpose, elle, seule.
3. **Redérivez la tension d'étalonnage pour votre propre distribution de litiges authentiques.** Ne réutilisez pas $\theta_0 = 4{,}0$ tiré du motif de renforcement d'un autre domaine ; relancez la procédure de la section 4 contre votre propre trafic.
4. **Relancez la suite adversariale, adaptée aux formes d'attaque de votre domaine.** Les cinq scénarios de l'article compagnon (salve en un pas, inondation persistante d'une même cohorte, inondation persistante à cohorte usurpée, pulvérisation sur de nombreux dossiers, campagnes parallèles synchronisées) forment une liste de départ raisonnable, non une liste close — réfléchissez à ce qu'un attaquant par épuisement de ressources tenterait effectivement en premier dans votre domaine précis.

8 Limites connues

La porte ne ferme pas l'attaque soutenue et diversifiée en identité ; elle la retarde, d'un facteur qui est lui-même un compromis réglable face à la barre imposée aux dossiers authentiques (corollaire 1 de l'article compagnon). Elle suppose que le proxy d'identité de cohorte a un coût réel à usurper ; si un attaquant peut générer une identité d'apparence structurellement fraîche à coût marginal nul à chaque pas, le facteur de retard constitue toute la défense, non un filet posé sur quelque chose de plus solide. La couche adaptative à la charge suppose que le signal de charge est câblé sur une occupation réelle, selon le second cas de dépannage ci-dessus — s'y tromper laisse le comportement de la couche indéfini par ce document. Enfin, cette porte est un contrôle d'admission, non un mécanisme de jugement sur le contenu : elle ne dit rien sur la question de savoir si un dossier, une fois admis, est correctement tranché par l'étage coûteux auquel elle donne accès.

Pour le modèle formel, les deux propositions et leurs preuves, le corollaire sur la tension d'étalonnage et la recherche d'antériorités, voir l'article compagnon de ce dossier.